

Implementação do Gerenciador de Memória e Escalonador Preemptivo em um Microkernel RISC-V

Isadora de Mello, Juliana Bertolazi Simon, Nicolas Vera

Disciplina de Sistemas Operacionais – Curso de Ciência da Computação – UNIVALI
Junho de 2026

1. Introdução

O projeto tem como objetivo a construção incremental do sistema operacional microkernel desenvolvido pelo professor Felipe Viel para a arquitetura RISC-V 64 bits, executado em modo supervisor sobre a plataforma de firmware OpenSBI e simulado pelo emulador QEMU. A base do projeto, provida pelo professor em documento e código de referência, já definia a estrutura geral do sistema: inicialização em Assembly, driver UART, criação de tarefas e chaveamento de contexto cooperativo.

O trabalho desenvolvido nesta etapa contemplou principalmente a evolução do alocador de memória linear (bump allocator) para um alocador baseado em lista livre com política First Fit. Além disso, foi realizada a implementação do escalonador preemptivo por timer, que permite a alternância automática entre tarefas sem que estas precisem ceder voluntariamente o processador via `yield()`.

Como contribuições adicionais não previstas na especificação mínima, foram implementados: o cálculo percentual de fragmentação externa do heap, um mapa visual da memória (`memory_dump`) e o desligamento automático do sistema após um número configurável de interrupções de timer.

2. Desenvolvimento

Esta seção descreve cada módulo implementado, as decisões de projeto tomadas e o raciocínio por trás das escolhas realizadas.

2.1. Gerenciador de Memória (Free List Allocator)

O alocador linear original avançava um ponteiro `heap_top` a cada chamada de `kmalloc()`, sem qualquer mecanismo de liberação ou reutilização. Isso implica em uma memória esgotada mesmo após o término de tarefas, o que inviabiliza cenários de criação e destruição dinâmica de tarefas em tempo de execução.

A implementação adotada substitui esse modelo por uma lista encadeada de blocos (free list). Cada bloco do heap possui um cabeçalho com três campos: tamanho útil do bloco (`size`), flag indicando se está livre (`free`) e ponteiro para o próximo bloco na lista (`next`). O heap é inicializado como um único bloco livre abrangendo toda a área disponível (64 KiB = 0x10000 bytes).

2.2. Política de Alocação: First Fit

A função `kmalloc()` percorre a lista encadeada em busca do primeiro bloco livre que possua tamanho suficiente para atender à requisição. Antes da busca, o tamanho solicitado é alinhado para múltiplos de 8 bytes, garantindo compatibilidade com a ABI do RISC-V 64 bits, que exige acesso alinhado à memória.

Quando o bloco encontrado é consideravelmente maior do que o necessário, a função `split_block()` é invocada para dividi-lo em dois: um bloco alocado ao usuário e um novo bloco livre com o espaço restante. A condição para que a divisão ocorra é que o espaço restante após a alocação seja suficiente para abrigar ao menos um cabeçalho adicional mais o alinhamento mínimo de 8 bytes — caso contrário, o desperdício interno (fragmentação interna) é aceito sem divisão, evitando a criação de blocos inúteis.

2.3. Desalocação e Coalescência

A função `kfree()` recebe um ponteiro retornado por `kmalloc()` e recupera o cabeçalho do bloco correspondente através de aritmética de ponteiros (o cabeçalho precede imediatamente os dados do usuário). O bloco é então marcado como livre (`free = 1`).

Após a marcação, a função `coalesce_blocks()` percorre toda a lista encadeada e une blocos livres adjacentes em um único bloco maior. Essa operação combate a fragmentação externa: sem coalescência, múltiplas liberações consecutivas resultariam em vários blocos pequenos separados por cabeçalhos, impedindo alocações maiores mesmo havendo espaço livre suficiente no total.

2.4. Escalonador Preemptivo Round-Robin

O escalonador implementado opera em dois modos complementares: cooperativo, via `yield()`, e preemptivo, via interrupções periódicas de timer. O modo cooperativo já estava esboçado na documentação de referência; o modo preemptivo foi integralmente desenvolvido nesta etapa.

A preempção é viabilizada por três componentes integrados: o módulo de timer (`timer.c`), o ponto de entrada de traps em Assembly (`trap_entry.S`) e o handler em C (`trap.c`). O timer é inicializado com um intervalo configurável e utiliza a chamada do `set_timer` para programar a próxima interrupção. A cada disparo, o trap handler identifica a causa e invoca `schedule_from_trap()`.

A função `schedule_from_trap()` realiza a troca de contexto preemptiva: ela copia o frame de registradores salvo por `trap_entry.S` para o TCB da tarefa atual, salva o valor do `sepc` (que contém o endereço da próxima instrução interrompida), seleciona a próxima tarefa via Round-Robin e substitui o conteúdo do frame pelos registradores da nova tarefa, ajustando também o `sepc`. Ao retornar com `sret`, a CPU restaura o contexto da nova tarefa a partir do frame modificado.

2.5. Ciclo de Vida das Tarefas

O módulo de tarefas foi estendido com um tipo enumerado `task_state_t` contendo os estados `TASK_FREE`, `TASK_READY` e `TASK_RUNNING`. O estado é atualizado pelo

scheduler a cada troca de contexto. A função `xTaskDelete()` foi implementada para liberar a stack de uma tarefa via `kfree()` e marcar o slot do TCB como disponível, permitindo a criação de novas tarefas naquele espaço.

O algoritmo Round-Robin foi ajustado para percorrer o vetor de TCBs e ignorar slots com `entry == 0` (tarefas destruídas ou não criadas), bem como slots no estado `TASK_FREE` — tornando o escalonador robusto a tarefas dinâmicas.

2.6. Implementações Extras

Além das funcionalidades previstas no enunciado, foram desenvolvidas duas implementações adicionais que contribuem para a validação do sistema.

Cálculo de Fragmentação Externa: a função `memory_fragmentation()` percorre a lista de blocos livres e calcula a razão entre o maior bloco livre individual e a soma total de memória livre, expressando a fragmentação como um percentual. A fórmula utilizada é:

$$\text{fragmentação} = 100 - (\text{maior_bloco_livre} * 100 / \text{total_livre})$$

Quando todos os blocos livres estão contíguos em um único bloco, o resultado é 0% (sem fragmentação). Quando a memória livre está dispersa em vários blocos pequenos, o percentual cresce, indicando degradação da disponibilidade efetiva de memória.

Mapa do Heap (`memory_dump`): a função `memory_dump()` percorre a lista encadeada e imprime via UART, para cada bloco, seu endereço físico, tamanho e status (LIVRE ou OCUPADO). Isso permite inspecionar visualmente o estado interno da heap durante a execução no QEMU, sendo fundamental para depurar cenários de fragmentação, split e coalescência.

Desligamento Automático: o trap handler contabiliza o número de interrupções de timer geradas. Após 20 alternâncias (configurável), o sistema escreve o código 0x5555 no endereço 0x100000 do QEMU, encerrando o emulador automaticamente. Isso permite executar testes completos com saída capturada em arquivo de log sem intervenção manual.

3. Testes

Os testes foram implementados diretamente na função `kernel_main()` do arquivo `main.c`, organizados em cinco etapas sequenciais que validam progressivamente as funcionalidades do gerenciador de memória, seguidas da demonstração do escalonador preemptivo.

3.1. Teste 1 – Estado Inicial do Heap

Objetivo: verificar que, após `memory_init()`, o heap é reportado como totalmente livre (65536 bytes), sem nenhum bloco alocado, e que a fragmentação é 0%. O mapa da heap

deve exibir um único bloco livre de 65512 bytes (diferença referente ao primeiro cabeçalho block_t).

```
=== VALIDACAO DO GERENCIADOR DE MEMORIA ===
Heap total: 65536 bytes
Heap usado: 0 bytes
Heap livre: 65536 bytes
Fragmentacao: 0%

===== MAPA HEAP =====
[Bloco 1] End: 0x2149599432 | Tam: 65512 bytes | Status: LIVRE [ . ]
=====
```

Figura 1. Saída do sistema após inicialização — heap totalmente livre, fragmentação 0%.

3.2. Teste 2 – Múltiplas Alocações e Split de Blocos

Objetivo: validar a alocação de três tarefas com tamanhos distintos (1024, 2048 e 1024 bytes) e confirmar que o mecanismo de split divide corretamente o bloco livre inicial em blocos menores. Após as alocações, o mapa deve exibir três blocos ocupados e um bloco livre residual, com fragmentação ainda em 0% (pois há um único bloco livre contíguo).

```
Task 0 criada (1024)
Task 1 criada (2048)
Task 2 criada (1024)
Heap total: 65536 bytes
Heap usado: 4168 bytes
Heap livre: 61368 bytes
Fragmentacao: 0%

===== MAPA HEAP =====
[Bloco 1] End: 0x2149599432 | Tam: 1024 bytes | Status: OCUPADO [ X ]
[Bloco 2] End: 0x2149600480 | Tam: 2048 bytes | Status: OCUPADO [ X ]
[Bloco 3] End: 0x2149602552 | Tam: 1024 bytes | Status: OCUPADO [ X ]
[Bloco 4] End: 0x2149603600 | Tam: 61344 bytes | Status: LIVRE [ . ]
=====
```

Figura 2. Mapa da heap com três alocações e um bloco livre residual ao final.

3.3. Teste 3 – Liberação de Memória (kfree)

Objetivo: validar a função kfree() através da remoção da Task 1 (bloco intermediário de 2048 bytes). Após a exclusão, o heap deve apresentar o bloco central marcado como LIVRE, enquanto os blocos adjacentes permanecem ocupados. A fragmentação deve agora ser maior que 0%, pois a memória livre está dispersa em dois blocos não contíguos.

```

Task 1 removida
Heap total: 65536 bytes
Heap usado: 2096 bytes
Heap livre: 63440 bytes
Fragmentacao: 4%

===== MAPA HEAP =====
[Bloco 1] End: 0x2149599432 | Tam: 1024 bytes | Status: OCUPADO [ X ]
[Bloco 2] End: 0x2149600480 | Tam: 2048 bytes | Status: LIVRE [ . ]
[Bloco 3] End: 0x2149602552 | Tam: 1024 bytes | Status: OCUPADO [ X ]
[Bloco 4] End: 0x2149603600 | Tam: 61344 bytes | Status: LIVRE [ . ]
=====

```

Figura 3. Mapa da heap após kfree() da Task 1 — bloco intermediário liberado, fragmentação não nula.

3.4. Teste 4 – Reutilização de Blocos (First Fit)

Objetivo: confirmar que uma nova alocação de 512 bytes (Task 3) reutiliza o bloco livre deixado pela Task 1, em vez de avançar para o final do heap. O bloco anteriormente de 2048 bytes deve ser dividido (split) em um bloco de 512 bytes ocupado e um bloco livre menor. Esse teste valida a política First Fit e a reutilização de memória — funcionalidades ausentes no bump allocator original.

```

Task 3 criada (512)
Heap total: 65536 bytes
Heap usado: 2632 bytes
Heap livre: 62904 bytes
Fragmentacao: 3%

===== MAPA HEAP =====
[Bloco 1] End: 0x2149599432 | Tam: 1024 bytes | Status: OCUPADO [ X ]
[Bloco 2] End: 0x2149600480 | Tam: 512 bytes | Status: OCUPADO [ X ]
[Bloco 3] End: 0x2149601016 | Tam: 1512 bytes | Status: LIVRE [ . ]
[Bloco 4] End: 0x2149602552 | Tam: 1024 bytes | Status: OCUPADO [ X ]
[Bloco 5] End: 0x2149603600 | Tam: 61344 bytes | Status: LIVRE [ . ]
=====

```

Figura 4. Task 3 (512 bytes) alocada no bloco liberado pela Task 1, com split do espaço excedente.

3.5. Teste 5 – Coalescência de Blocos Adjacentes

Objetivo: validar a coalescência removendo todas as tarefas restantes (Tasks 0, 2 e 3) e verificando que os blocos livres adjacentes são fundidos em um único bloco grande. Após a operação, o heap deve retornar a um estado próximo ao inicial — um único bloco livre — com fragmentação de 0%. Esse teste comprova que o sistema é capaz de recuperar completamente a memória.

```

Task 0 removida
Task 1 removida
Task 2 removida
Task 3 removida
Heap total: 65536 bytes
Heap usado: 0 bytes
Heap livre: 65536 bytes
Fragmentacao: 0%

===== MAPA HEAP =====
[Bloco 1] End: 0x2149599432 | Tam: 65512 bytes | Status: LIVRE [ . ]
=====

```

Figura 5. Heap após coalescência total — um único bloco livre, fragmentação 0%.

3.6. Teste 6 – Escalonador Preemptivo Round-Robin

Objetivo: demonstrar o funcionamento do escalonador preemptivo. Duas tarefas (task1 e task2) são criadas e o escalonador é iniciado. Nenhuma das tarefas chama yield(); a alternância entre elas ocorre exclusivamente via interrupções de timer. A saída esperada é a impressão alternada das mensagens de cada tarefa, terminando o programa com o desligamento automático após 20 ticks.

```

Iniciando o Escalonador Preemptivo Round-Robin...
[TASK 1] Processando via preempcao de hardware...
[TASK 1] Processando via preempcao de hardware...
[TASK 1] Processando via preempcao de hardware...
[TASK 1] Processando via preempcao de hardware...
[TASK 1] Processando via preempcao de hardware...
[TASK 1] Processando via preempcao de hardware...
[TASK 1] Processando via preempcao de hardware...
[TASK 2] Processando via preempcao de hardware...
[TASK 2] Processando via preempcao de hardware...
[TASK 2] Processando via preempcao de hardware...
[TASK 2] Processando via preempcao de hardware...
[TASK 2] Processando via preempcao de hardware...
[TASK 2] Processando via preempcao de hardware...
[TASK 2] Processando via preempcao de hardware...
[TASK 2] Processando via preempcao de hardware...
[TASK 2] Processando via preempcao de hardware...
[TASK 2] Processando via preempcao de hardware...

```

Figura 6. Saída do escalonador preemptivo — alternância entre tarefas sem uso de yield().

4. Conclusão

O desenvolvimento deste trabalho permitiu uma compreensão aprofundada dos mecanismos internos de um sistema operacional. A implementação do Free List Allocator

demonstrou na prática os conceitos de fragmentação, coalescência, split de blocos e políticas de alocação.

O escalonador preemptivo introduziu um grau de complexidade significativamente maior: a interação entre o código em Assembly (`trap_entry.S` e `context.S`) do RISC-V exigiu uma compreensão cuidadosa do fluxo de execução durante uma interrupção. Pequenos erros no mapeamento dos offsets do frame de registradores resultavam em comportamentos imprevisíveis — como tarefas que nunca chegavam a executar ou que corrompiam o stack pointer.

As principais dificuldades encontradas ao longo do projeto foram:

- Interação com Assembly: a compreensão dos registradores de controle e status do RISC-V (`scause`, `sepc`, `stvec`, `sstatus`, `sie`) e do seu papel no tratamento de traps exigiu estudo detalhado sobre o RISC-V.
- Mapeamento do frame no context switch preemptivo: a função `schedule_from_trap()` precisa copiar registradores entre o frame salvo em Assembly e o vetor `regs[]` do TCB, respeitando exatamente a ordem definida em `trap_entry.S` e `context.S`. Um mismatch nesse mapeamento é difícil de depurar pois se manifesta como comportamento incorreto da tarefa restaurada, não como um crash imediato.
- Compreensão do sistema sem experiência prévia: o projeto exigiu que conceitos como modo supervisor, pilha de kernel e convenções de chamada fossem absorvidos simultaneamente, sem uma curva de aprendizado gradual. A leitura da documentação do projeto foi essencial para reduzir esse impacto.
- Depuração em ambiente bare-metal: sem um sistema de arquivos, sem um depurador funcional de alto nível e sem mensagens de erro do sistema operacional, qualquer falha se manifesta como trava silenciosa ou comportamento errático. O uso de `uart_print()` como ferramenta de depuração foi indispensável.

Apesar das dificuldades, os resultados obtidos validam a abordagem adotada. O gerenciador de memória opera corretamente em todos os cenários testados — incluindo a reutilização de blocos via First Fit, a divisão de blocos excedentes e a fusão de blocos adjacentes liberados. O escalonador preemptivo demonstra alternância automática entre tarefas, confirmando que o sistema opera em modo verdadeiramente preemptivo.